

DB FINAL PAST PAPER

December 9, 2025

Given Table

Question 1: Students can lease university flats and some of the details of leases held by students for places in university flats are shown below in table. A place number (placeNo) uniquely identifies each single room in all flats and is used when leasing a room to a student. BannerID is used to uniquely identify a student. [7 marks]

leaseNo	bannerID	placeNo	fName	lName	startDate	finishDate	flatNo	flatAddress
10003	B017706	78	Jane	Watt	01/09/2010	30/06/2011	F56	34 High Street, Paisley
10259	B017706	88	Jane	Watt	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
10364	B013399	89	Tom	Jones	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
10566	B012124	102	Karen	Black	01/09/2011	30/06/2012	F79	120 Lady Lane, Paisley
11067	B034511	88	Steven	Smith	01/09/2012	30/06/2013	F78	111 Storrie Road, Paisley
11169	B013399	78	Tom	Jones	01/09/2012	30/06/2013	F56	34 High Street, Paisley

- The table shown above is susceptible to update anomalies. Provide examples of insertion, deletion, and update anomalies.
- Describe and illustrate the process of normalizing the table shown above to 3NF, by identifying the functional dependencies represented by the attributes. State any assumptions you make about the data shown in the table.

Figure 1: Q1

(a) Update Anomalies

1. Insertion Anomaly: If a new flat is created (e.g., flatNo F90, address known), but no student has leased a room yet, we cannot insert its address because the table requires a *leaseNo* and student details.

2. Deletion Anomaly: If the last lease for flat F78 expires and is deleted, we also lose the flat's address *111 Storrie Road, Paisley*. Flat information should not depend on having a lease.

3. Update Anomaly: If flat F78 changes address, we must update it in multiple rows. If one row is missed, inconsistencies occur.

(b) Normalization to 3NF

Functional Dependencies (Assumptions Stated)

- $\text{leaseNo} \rightarrow \text{bannerID}, \text{placeNo}, \text{startDate}, \text{finishDate}$
- $\text{bannerID} \rightarrow \text{fName}, \text{lName}$
- $\text{placeNo} \rightarrow \text{flatNo}$
- $\text{flatNo} \rightarrow \text{flatAddress}$

Primary Key

For the original table:

Primary Key = leaseNo

Step 1: First Normal Form (1NF)

Requirement: No repeating groups; atomic values.

The table is already in 1NF since all attributes contain atomic values.

Step 2: Second Normal Form (2NF)

Requirement: No partial dependencies on a composite key. Since the key is a *single attribute* (leaseNo), 2NF is automatically satisfied. However, many non-key attributes depend on other non-key attributes, causing transitive dependencies.

Step 3: Third Normal Form (3NF)

Requirement: Remove transitive dependencies.

Transitive dependencies:

$\text{leaseNo} \rightarrow \text{bannerID} \rightarrow \text{studentName}$

$\text{leaseNo} \rightarrow \text{placeNo} \rightarrow \text{flatNo} \rightarrow \text{flatAddress}$

Decomposed Relations (3NF)

1. Student Table

Student(bannerID, fName, lName)

2. Flat Table

Flat(flatNo, flatAddress)

3. Room Table

Room(placeNo, flatNo)

4. Lease Table

Lease(leaseNo, bannerID, placeNo, startDate, finishDate)

All relations now satisfy 3NF:

- No transitive dependencies
- All attributes depend only on their primary key

Normalization Theory Summary

1NF: Data is atomic; no repeating groups.

2NF: No partial dependency on a composite key.

3NF: No transitive dependencies; every non-key attribute depends directly on the primary key.

BCNF (Boyce–Codd Normal Form): For every functional dependency $X \rightarrow Y$, X must be a superkey. BCNF is stronger than 3NF.

Question 2: Relational Algebra

Given Schema

Branch(branchNo, street, city, postcode)

Staff(staffNo, fName, lName, position, sex, DOB, salary, branchNo)

PropertyForRent(propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)

Client(clientNo, fName, lName, telNo, prefType, maxRent, eMail)

PrivateOwner(ownerNo, fName, lName, address, telNo, eMail, password)

Viewing(clientNo, propertyNo, viewDate, comment)

Registration(clientNo, branchNo, staffNo, dateJoined)

(a) How many properties cost more than £350 per month to rent?

Relational Algebra

$\gamma_{\text{count(propertyNo)} \rightarrow \text{totalProperties}}(\sigma_{\text{rent} > 350}(\text{PropertyForRent}))$

Explanation (Beginner friendly):

- $\sigma_{\text{rent} > 350}$ selects (filters) only properties with rent greater than 350.
- γ is a grouping/aggregation operator.

- We count the number of selected properties.

SQL Equivalent

```
SELECT COUNT(propertyNo) AS totalProperties
FROM PropertyForRent
WHERE rent > 350;
```

(b) Identify all clients who have viewed all properties with three rooms.

Relational Algebra

$$ThreeRoomProps \leftarrow \pi_{propertyNo}(\sigma_{rooms=3}(PropertyForRent))$$

$$ClientViewed \leftarrow \pi_{clientNo,propertyNo}(Viewing)$$

$$Result \leftarrow ClientViewed \div ThreeRoomProps$$

Explanation (Beginner friendly):

- First line: Select all properties that have exactly 3 rooms and keep only their property numbers.
- Second line: Take the (client, property) pairs from Viewing.
- Third line: The division operator ($\div$) finds clients who have viewed **every** 3-room property.

What does division mean? (Simple Explanation) Division answers questions of the form: “Find X who have done ALL of Y.”

Example:

- Y = all 3-room properties
- X = clients

Division checks for each client whether they have viewed **all items in Y**. If yes $\rightarrow$ client is included in the result.

SQL Equivalent

```
SELECT c.clientNo
FROM Client c
WHERE NOT EXISTS (
    SELECT p.propertyNo
    FROM PropertyForRent p
    WHERE p.rooms = 3
    EXCEPT
```

```

SELECT v.propertyNo
FROM Viewing v
WHERE v.clientNo = c.clientNo
);

```

Relational Algebra (without division)

$$ThreeRoomProps \leftarrow \pi_{propertyNo}(\sigma_{rooms=3}(PropertyForRent))$$

$$ClientViewed \leftarrow \pi_{clientNo,propertyNo}(Viewing)$$

$$Result \leftarrow \pi_{clientNo}(ClientViewed) - \pi_{clientNo}(ClientViewed \bowtie (ThreeRoomProps - \pi_{propertyNo}(ClientViewed)))$$

Explanation (Beginner Friendly):

- Get all properties with 3 rooms.
- For each client, check whether there exists a 3-room property they **did NOT** view.
- If a client has missing views, exclude them.
- Final result = clients who did **not miss any** 3-room property.

SQL Equivalent (Simple Nested Subquery — No Division)

```

SELECT DISTINCT v.clientNo
FROM Viewing v
WHERE NOT EXISTS (
    SELECT p.propertyNo
    FROM PropertyForRent p
    WHERE p.rooms = 3
    AND p.propertyNo NOT IN (
        SELECT v2.propertyNo
        FROM Viewing v2
        WHERE v2.clientNo = v.clientNo
    )
);

```

Beginner Explanation:

- Outer query: go client by client.
- Inner query: find 3-room properties.
- Condition checks:

Is there ANY 3-room property this client did NOT view?

- If such a property exists $\rightarrow$ client fails.
- We keep clients where the answer is NO, using:

WHERE NOT EXISTS

Relational Algebra (Using Division)

$$ThreeRoomProps \leftarrow \pi_{propertyNo}(\sigma_{rooms=3}(PropertyForRent))$$

$$ClientViewed \leftarrow \pi_{clientNo,propertyNo}(Viewing)$$

$$Result \leftarrow ClientViewed \div ThreeRoomProps$$

Explanation (Beginner friendly):

- First line: Select all properties that have exactly 3 rooms and keep only their property numbers.
- Second line: Take the (client, property) pairs from Viewing.
- Third line: The division operator ($\div$) finds clients who have viewed **every** 3-room property.

SQL Equivalent (Using Nested Subquery / NOT EXISTS)

```
SELECT DISTINCT v.clientNo
FROM Viewing v
WHERE NOT EXISTS (
    SELECT p.propertyNo
    FROM PropertyForRent p
    WHERE p.rooms = 3
    AND p.propertyNo NOT IN (
        SELECT v2.propertyNo
        FROM Viewing v2
        WHERE v2.clientNo = v.clientNo
    )
);
```

SQL Equivalent (Using GROUP BY and HAVING)

```
SELECT v.clientNo
FROM Viewing v
JOIN PropertyForRent p ON v.propertyNo = p.propertyNo
WHERE p.rooms = 3
GROUP BY v.clientNo
HAVING COUNT(DISTINCT v.propertyNo) =
```

```
(
  SELECT COUNT(*)
  FROM PropertyForRent
  WHERE rooms = 3
);
```

Explanation (GROUP BY + HAVING version):

- Join Viewing with PropertyForRent to consider only 3-room properties.
- Group by each client.
- Use HAVING to ensure the client has viewed **all** 3-room properties (count matches total 3-room properties).

(c) List the names and comments of all clients who have viewed a property for rent.

Relational Algebra

$$Result \leftarrow \pi_{Client.fName, Client.lName, Viewing.comment}(Client \bowtie_{Client.clientNo=Viewing.clientNo} Viewing)$$

Explanation (Beginner friendly):

- We join Client and Viewing tables using clientNo.
- The join matches each client with their viewing records.
- Then we project (select) only: first name, last name, and comment.

SQL Equivalent

```
SELECT c.fName, c.lName, v.comment
FROM Client c
JOIN Viewing v ON c.clientNo = v.clientNo;
```

(d) List all cities where there is either a branch office or a property for rent.

Relational Algebra

$$Cities \leftarrow \pi_{city}(Branch) \cup \pi_{city}(PropertyForRent)$$

Explanation (Beginner friendly):

- We project (select) just the city from Branch.
- We also project city from PropertyForRent.
- The union $\cup$ combines both lists without duplicates.

SQL Equivalent

```
SELECT city FROM Branch
UNION
SELECT city FROM PropertyForRent;
```

(e) List all cities where there is a branch office but no properties for rent.

Relational Algebra

$$Result \leftarrow \pi_{city}(Branch) - \pi_{city}(PropertyForRent)$$

Explanation (Beginner friendly):

- Projection on Branch gives all cities with a branch.
- Projection on PropertyForRent gives all cities having properties.
- Set difference ($-$) removes all cities that appear in PropertyForRent.
- Remaining cities have branches but **no** properties for rent.

SQL Equivalent

```
SELECT city
FROM Branch
MINUS
SELECT city
FROM PropertyForRent;
```

Question 3: Conceptual Data Model

The company specializes in IT training and has the following requirements:

- 30 instructors.
- Up to 100 trainees per training session.
- Five advanced technology courses.
- Each course is taught by a teaching team of two or more instructors.
- Each instructor is assigned to a maximum of two teaching teams **or** may be assigned to conduct research.
- Each trainee undertakes exactly one advanced technology course per training session.

Based on the requirements, the main **entity types**, their **attributes**, **primary keys**, **foreign keys**, and **relationships with multiplicities** are modeled below.

Entities and Attributes

- **INSTRUCTOR** (InstructorID, Name, Expertise)
- **COURSE** (CourseID, CourseName, Duration)
- **TEACHINGTEAM** (TeamID, CourseID(FK))
- **TRAINEE** (TraineeID, Name, Email)
- **TRAININGSESSION** (SessionID, Location, Capacity)
- **RESEARCH** (ResearchID, Topic, Description)

Relationships and Structural Constraints

- **AssignedTo:** INSTRUCTOR — TEACHINGTEAM
 - Each instructor is assigned to 0–2 teaching teams.
 - Each teaching team must have 2–n instructors.
- **Conducts:** INSTRUCTOR — RESEARCH
 - An instructor may conduct 0–n research projects.
 - Each research project must be conducted by at least 1 instructor.
- **TaughtBy:** COURSE — TEACHINGTEAM
 - Each course is taught by exactly 1 teaching team.
 - Every teaching team teaches exactly 1 course.
- **RegistersFor:** TRAINEE — TRAININGSESSION
 - Each trainee registers for exactly 1 training session.
 - Each training session can have 0–100 trainees.
- **Covers:** TRAININGSESSION — COURSE
 - Each training session covers exactly 1 course.
 - A course may have multiple training sessions (0–n).

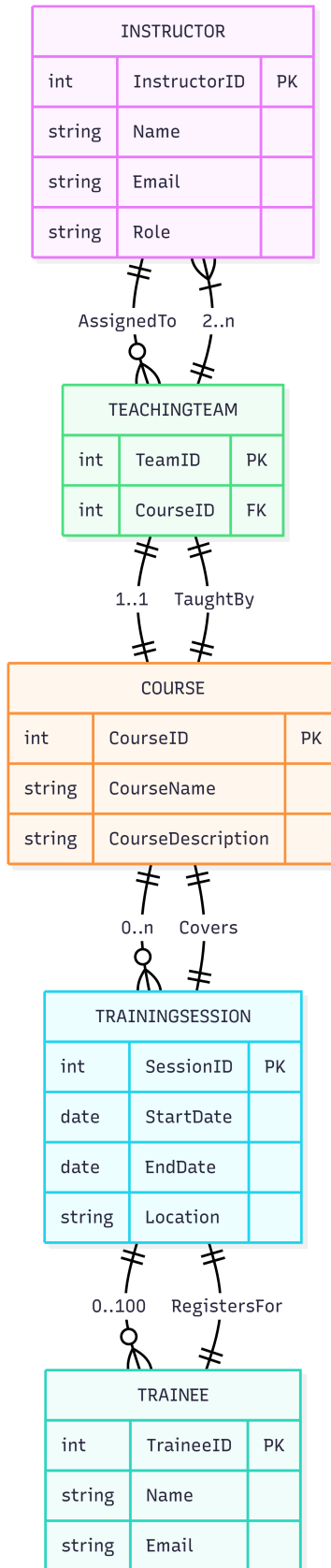


Figure 2: ERD

ER Diagram

Summary

The above conceptual data model fully satisfies the requirements by:

- Representing all main entities with their key attributes.
- Capturing all structural constraints such as:
 - Instructor max 2 teaching teams.
 - Teaching team minimum of 2 instructors.
 - Each trainee attends exactly one session.
 - Sessions limited to 100 trainees.
 - Research handled as a separate relationship.
- Providing appropriate primary and foreign keys.
- Giving an ER diagram with correct cardinalities.

Question 5 (15 marks)

Given:

Transaction T_1 : $R_1(x)$, $W_1(x)$, $R_1(y)$, $W_1(y)$.

Transaction T_2 : $R_2(x)$, $W_2(x)$.

Only operations on the same data item by different transactions cause conflicts. Here only item x is shared; operations on y do not conflict with T_2 . We therefore extract the order of x -operations from each schedule and build the precedence graph: an edge $T_i \rightarrow T_j$ exists if an operation of T_i on x precedes a conflicting operation of T_j on x . A cycle in the graph $\Rightarrow$ not conflict-serializable. A single directed edge $T_i \rightarrow T_j$ (no reverse edge) means conflict-equivalent to the serial order T_i then T_j .

S1 $R_1(x)$; $W_1(x)$; $R_1(y)$; $W_1(y)$; $R_2(x)$; $W_2(x)$

Order of x-ops: R_1 , W_1 , R_2 , W_2 .

Conflicting edges: W_1 before R_2 and $W_2 \Rightarrow$ edges $T_1 \rightarrow T_2$ only.

Precedence graph: $T_1 \rightarrow T_2$. No cycle.

Classification: *Serial* (exactly T_1 then T_2); hence conflict-serializable.

S2 $R_2(x)$; $R_1(x)$; $W_2(x)$; $W_1(x)$; $R_1(y)$; $W_1(y)$

Order of x-ops: R_2 , R_1 , W_2 , W_1 .

Conflicts: R_1 before W_2 gives $T_1 \rightarrow T_2$. R_2 before W_1 gives $T_2 \rightarrow T_1$.

Precedence graph: $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ (cycle).

Classification: *Not conflict-serializable*.

- S3** $R_1(x); W_1(x); R_1(y); R_2(x); W_1(y); W_2(x)$
Order of x-ops: R_1, W_1, R_2, W_2 .
Conflicts: W_1 before $R_2, W_2 \Rightarrow T_1 \rightarrow T_2$ only.
Precedence graph: $T_1 \rightarrow T_2$. No cycle.
Classification: *Conflict-serializable*, equivalent to serial order T_1 then T_2 (but not a serial schedule).
- S4** $R_2(x); R_1(x); W_1(x); R_1(y); W_1(y); W_2(x)$
Order of x-ops: R_2, R_1, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle $T_1 \leftrightarrow T_2$.
Classification: *Not conflict-serializable*.
- S5** $R_1(x); W_1(x); R_1(y); R_2(x); W_2(x); W_1(y)$
Order of x-ops: R_1, W_1, R_2, W_2 .
Conflicts: W_1 before $R_2, W_2 \Rightarrow T_1 \rightarrow T_2$ only.
Precedence graph: $T_1 \rightarrow T_2$. No cycle.
Classification: *Conflict-serializable* (equivalent to T_1 then T_2).
- S6** $R_1(x); W_1(x); R_2(x); R_1(y); W_2(x); W_1(y)$
Order of x-ops: R_1, W_1, R_2, W_2 .
Conflicts: W_1 before $R_2, W_2 \Rightarrow T_1 \rightarrow T_2$ only.
Precedence graph: $T_1 \rightarrow T_2$. No cycle.
Classification: *Conflict-serializable* (equivalent to T_1 then T_2).
- S7** $R_2(x); R_1(x); W_1(x); R_1(y); W_2(x); W_1(y)$
Order of x-ops: R_2, R_1, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.
- S8** $R_1(x); W_1(x); R_2(x); W_2(x); R_1(y); W_1(y)$
Order of x-ops: R_1, W_1, R_2, W_2 .
Conflicts: W_1 before $R_2, W_2 \Rightarrow T_1 \rightarrow T_2$ only.
Precedence graph: $T_1 \rightarrow T_2$. No cycle.
Classification: *Conflict-serializable* (equivalent to T_1 then T_2).
- S9** $R_1(x); R_2(x); W_1(x); R_1(y); W_1(y); W_2(x)$
Order of x-ops: R_1, R_2, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.

- S10** $R_1(x); R_2(x); W_1(x); R_1(y); W_2(x); W_1(y)$
Order of x-ops: R_1, R_2, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.
- S11** $R_1(x); R_2(x); W_1(x); W_2(x); R_1(y); W_1(y)$
Order of x-ops: R_1, R_2, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.
- S12** $R_1(x); W_1(x); R_2(x); R_1(y); W_1(y); W_2(x)$
Order of x-ops: R_1, W_1, R_2, W_2 .
Conflicts: W_1 before R_2 and $W_2 \Rightarrow T_1 \rightarrow T_2$ only.
Precedence graph: $T_1 \rightarrow T_2$. No cycle.
Classification: *Conflict-serializable* (equivalent to T_1 then T_2).
- S13** $R_1(x); R_2(x); W_2(x); W_1(x); R_1(y); W_1(y)$
Order of x-ops: R_1, R_2, W_2, W_1 .
Conflicts: R_1 before W_2 gives $T_1 \rightarrow T_2$. R_2 before W_1 gives $T_2 \rightarrow T_1$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.
- S14** $R_2(x); R_1(x); W_1(x); W_2(x); R_1(y); W_1(y)$
Order of x-ops: R_2, R_1, W_1, W_2 .
Conflicts: R_2 before W_1 gives $T_2 \rightarrow T_1$. R_1 before W_2 gives $T_1 \rightarrow T_2$.
Precedence graph: cycle.
Classification: *Not conflict-serializable*.
- S15** $R_2(x); W_2(x); R_1(x); W_1(x); R_1(y); W_1(y)$
Order of x-ops: R_2, W_2, R_1, W_1 .
Conflicts: all x -ops of T_2 occur before any x -ops of $T_1 \Rightarrow$ edges $T_2 \rightarrow T_1$ only.
Precedence graph: $T_2 \rightarrow T_1$. No cycle.
Classification: *Serial* (exactly T_2 then T_1); hence conflict-serializable.

Summary table:

- **Serial schedules:** S1 (T_1 then T_2), S15 (T_2 then T_1).
- **Conflict-serializable but not serial:** S3, S5, S6, S8, S12. (All have precedence graph $T_1 \rightarrow T_2$.)

- **Not conflict-serializable (cycles in precedence graph):** S2, S4, S7, S9, S10, S11, S13, S14.

Notes / reasoning tips:

- Only operations on the same data item x between different transactions matter for conflict-serializability here; y is private to T_1 .
- Build the precedence graph from the order of conflicting x -operations: if an operation of T_i on x appears before a conflicting operation of T_j on x , add edge $T_i \rightarrow T_j$.
- If the graph has a cycle (both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$), then the schedule is not conflict-serializable.
- If the graph has a single directed edge (no cycle), the schedule is conflict-equivalent to the corresponding serial order.

Question 6

You are designing a database for KW Humane Society. The result is the following set of relations:

- **Animals**(ID: integer, Name: string, PrevOwner: string, DateAdmitted: date, Type: string)
- **Adopter**(SIN: integer, Name: string, Address: string, OtherAnimals: integer)
- **Adoption**(AnimalID: integer, SIN: integer, AdoptDate: date, chipNo: integer)

The following are the required SQL queries.

(a) Retrieve the total number of dogs that were brought to the Humane Society on 18 April 2000.

```
SELECT COUNT(*)
FROM Animals
WHERE Type = 'dog'
AND DateAdmitted = '2000-04-18';
```

(b) List the name of the adopter who has adopted every type of animal.

```
SELECT A.Name
FROM Adopter A
JOIN Adoption Ad ON A.SIN = Ad.SIN
JOIN Animals An ON Ad.AnimalID = An.ID
GROUP BY A.SIN, A.Name
HAVING COUNT(DISTINCT An.Type) = (
    SELECT COUNT(DISTINCT Type)
    FROM Animals
);
```

(c) For each animal type, list the animal type and total number of adoptions on 14 June 1999.

```
SELECT An.Type, COUNT(*) as TotalAdoptions
FROM Animals An
JOIN Adoption Ad ON An.ID = Ad.AnimalID
WHERE Ad.AdoptDate = '1999-06-14'
GROUP BY An.Type;
```

(d) List the types of animals who have not had any adoptions.

```
SELECT DISTINCT Type
FROM Animals
WHERE Type NOT IN (
    SELECT An.Type
    FROM Animals An
    JOIN Adoption Ad ON An.ID = Ad.AnimalID
);
```

(e) For each adopter who has made at least two adoptions, list their names and addresses.

```
SELECT A.Name, A.Address
FROM Adopter A
JOIN Adoption Ad ON A.SIN = Ad.SIN
GROUP BY A.SIN, A.Name, A.Address
HAVING COUNT(Ad.AnimalID) >= 2;
```